

Lab Page Tables 实验报告

李睿阳 524120910059

1. Subtask 实现思路、修改文件及关键代码

Task 1: Speed up system calls (ugetpid)

实现思路：

通过在内核与用户空间之间共享一个只读页来加速 `getpid()` 系统调用。在进程创建时，分配一个物理页并建立从虚拟地址 `USYSCALL` 到该物理页的映射，在该页开头存储进程的 PID。

修改文件：

- [kernel/proc.h](#)
- [kernel/proc.c](#)

关键代码：

1. 在 `struct proc` 中增加字段：

```
struct proc {  
    // ...  
    struct usyscall* usyscall;  
};
```

2. 在 `allocproc` 中分配并初始化该页：

```
if((p->usyscall = (struct usyscall *)kalloc()) == 0){  
    freeproc(p);  
    release(&p->lock);  
    return 0;  
}  
p->usyscall->pid = p->pid;
```

3. 在 `proc_pagetable` 中建立页表映射：

```

if(mappages(pagetable, USYSCALL, PGSIZE,
            (uint64)(p->usyscall), PTE_U | PTE_R) < 0){
    // ... 错误处理
}

```

4. 在 freeproc 中释放物理页:

```

if(p->usyscall)
    kfree((void*)p->usyscall);

```

5. 在 ulib.c 中的函数实现:

```

int
ugetpid(void)
{
    struct usyscall *u = (struct usyscall *)USYSCALL;
    return u->pid;
}

```

Task 2: Print a page table (vmprint)

实现思路:

实现一个递归函数 `_vmprint` 来遍历三级页表结构，打印有效（PTE_V）的条目。根据深度缩进，并打印 PTE 索引、PTE 内容以及对应的物理地址。

修改文件:

- [kernel/vm.c](#)
- [kernel/defs.h](#)
- [kernel/exec.c](#)

关键代码:

1. 递归遍历打印页表 (kernel/vm.c):

```

void _vmprint(pagetable_t pagetable, uint64 level) {
    for(int i = 0; i < 512; i++){
        pte_t pte = pagetable[i];
        if(pte & PTE_V){
            // 根据层级打印缩进 " .."
            for (int j = 0; j <= level; j++) printf(" ..");
            uint64 pa = PTE2PA(pte);
            printf("%d: pte %p pa %p\n", i, pte, pa);
            // 如果是非叶子节点（且 valid），继续递归
            if ((pte & (PTE_R | PTE_W | PTE_X)) == 0) {
                _vmprint((pagetable_t)pa, level + 1);
            }
        }
    }
}

```

2. 在 exec.c 中调用：

```

if(p->pid==1) vmprint(p->pagetable);

```

Task 3: Detect which pages have been accessed (pgaccess)

实现思路：

实现 pgaccess 系统调用。根据输入的起始地址和页面数量，使用 walk 函数查找对应的页表项（PTE），检查 PTE_A（访问位）是否被设置。如果设置了，则记录到结果位掩码中，并清空该位以备后续检测。最后将结果拷贝回用户空间。

修改文件：

- [kernel/riscv.h](#)
- [kernel/sysproc.c](#)

关键代码：

1. 定义 PTE_A（kernel/riscv.h）：

```

#define PTE_A (1L << 6) // access bit

```

2. 实现系统调用逻辑（kernel/sysproc.c）：

```

int sys_pgaccess(void) {
    uint64 base;
    int len;
    uint64 user_mask_addr;
    uint64 kernel_mask = 0;
    argaddr(0, &base);
    argint(1, &len);
    argaddr(2, &user_mask_addr);

    for (int i = 0; i < len; i++) {
        uint64 va = base + i * PGSIZE;
        pte_t *pte = walk(myproc()->pagetable, va, 0);
        if(pte && (*pte & PTE_A)) {
            kernel_mask |= (1 << i);
            *pte &= ~PTE_A; // 清除访问位
        }
    }
    return copyout(myproc()->pagetable, user_mask_addr, (char *)&kernel_mask, sizeof(kernel_m
}

```

2. 调试中的要点与注意事项

1. 权限位设置：

在 Task 1 中，共享页在页表中必须设置 PTE_U（用户可访问）和 PTE_R（只读）。如果忘记 PTE_U，用户态 `ugetpid()` 会触发缺页中断；如果设置了 PTE_W，在只读安全性层面有风险。

2. 页表释放的完整性：

在 `freeproc` 中，不仅要释放 `usyscall` 的物理内存，还要确保在 `proc_freepagetable` 中解除相关的映射。如果不释放物理页，会导致内存泄漏。

3. 递归结束条件：

在 `vmprint` 递归中，区分叶子节点和非叶子节点非常重要。RISC-V 规定如果 PTE_R/W/X 均为 0，则该 PTE 指向下一级页表，否则是叶子页面。

4. 清除访问位：

在检测后必须清除 PTE_A，否则该位一经触发将永远保持为 1，无法检测后续的访问。

5. 关于 Task 3 实现的思考：

- **为什么 Task 3 不能像 Task 1 那样使用快捷方式（shortcut）：**Task 1 的 `ugetpid` 共享的是一个固定的、由内核直接更新的信息（PID）。而 Task 3 的 `pgaccess` 需要检查页表项的 PTE_A

位，这些位是由硬件（MMU）在进程运行过程中动态设置在页表里的。用户态程序无法安全且实时地直接访问页表的内部结构（PTEs），因此必须通过内核态读取。

- **为什么一定要在内核态执行：**页表是受保护的内核数据结构。为了安全，用户态进程只能访问特定的映射页面，不能随意遍历整个页表结构。只有进入内核态，才有权限通过 `walk` 函数查找任意虚拟地址对应的 PTE。
- **为什么需要实现 `copyout`：**因为 `pgaccess` 的检测结果（掩码）需要返回给用户进程提供的缓冲区。内核与用户空间使用不同的页表，内核不能直接对用户态指针进行解引用赋值，必须使用 `copyout` 来完成跨地址空间的数据传输。
- **关于 `PTE_A` 的赋值：**不需要手动管理给 `PTE_A` 赋值为 1。当 RISC-V 硬件 MMU 访问某个页面时，会自动将对应 PTE 中的 `PTE_A` 位置位。我们的任务是在检测到后将其“清零”，由硬件负责“置一”。